CITY ENGINEERING COLLEGE

VISVESVARAYA TECHNOLOGICAL UNIVERSITY BELAGAVI

Computer Science and Engineering

LAB MANUAL

Web Technology Lab

Subject Code: BCSL504

Name:
USN:
Semester/Section:

	EXPERIMENTS
1.	**Develop the HTML page named as "Myfirstwebpage.html". Add the following tags with relevant content.** **1. Set the title of the page as "My First Web Page"** **2. Within the body use the following tags:** **a) Moving text = "Basic HTML Tags"** **b) Different heading tags (h1 to h6)** **c) Paragraph** **d) Horizontal line** **e) Line Break** **f) Block Quote** **g) Pre tag** **h) Different Logical style (<b>, <u>, <sub>, <sup> etc.)**
2.	**Develop the HTML page named as "Table.html" to display your class time table. a) Provide the title as Time Table with table header and table footer, row-span and col-span etc.** **b) Provide various colour options to the cells (Highlight the lab hours and elective hours with different colours.)** **c) Provide colour options for rows.**
3.	**Develop an external style sheet named as "style.css" and provide different styles for h2, h3, hr, p, div, span, time, img & a tags. Apply different CSS selectors for tags and demonstrate the significance of each. 4 Develop HTML page named as "registration.html" having variety of HTML input elements with background**
4.	**Develop HTML page named as "registration.html" having variety of HTML input elements with background**
5.	**Develop HTML page named as "newpaper.html" having variety of HTML semantic elements with background colors, text-colors & size for figure, table, aside, section, article, header, footer… etc.**
6.	**Apply HTML, CSS and JavaScript to design a simple calculator to perform the following operations: sum, product, difference, remainder, quotient, power, square-root and square.**
7.	**Develop JavaScript program (with HTML/CSS) for:** **a) Converting JSON text to JavaScript Object** **b) Convert JSON results into a date** **c) Converting From JSON To CSV and CSV to JSON** **d) Create hash from string using crypto.createHash() method**
8.	**a. Develop a PHP program (with HTML/CSS) to keep track of the number of visitors visiting the web page and to display this count of visitors, with relevant headings.** **b. Develop a PHP program (with HTML/CSS) to sort the student records which are stored in the database using selection sort.**
9.	**Develop jQuery script (with HTML/CSS) for:** **a. Appends the content at the end of the existing paragraph and list.** **b. Change the state of the element with CSS style using animate() method** **c. Change the color of any div that is animated.**
10	**Develop a JavaScript program with Ajax (with HTML/CSS) for:** **a. Use ajax() method (without Jquery) to add the text content from the text file by sending ajax request.** **b. Use ajax() method (with Jquery) to add the text content from the text file by sending ajax request.** **c. Illustrate the use of getJSON() method in jQuery** **d. Illustrate the use of parseJSON() method to display JSON values.**

CSE/CEC

WEB TECHNOLOGY LAB [BCSL504]

Course objectives:

1. Learn HTML 5 elements and their use.
2. Use of CSS for enhanced user interface presentation.
3. Gain knowledge of JavaScript, AJAX and jQuery for dynamic presentation.
4. Use of PHP to build Web applications.
5. Design and develop Websites and Web applications

Programming Assignment (5 marks):

Construct a website (multiple Web pages) containing 'Resume' and Bio -data by using relevant HTML elements and appropriate styling for presentation with CSS/jQuery/JavaScript. Host the Website on a cloud platform.

Programming Assignment (5 marks):

Build a Web application with HTML, CSS, JavaScript, jQuery and PHP for online application/registration form. Form should accept the information and print/display on a browser with formatting/styling upon submission (Button click) on success. Host the application on a cloud platform.

Course outcomes (Course Skill Set):

At the end of the course, the student will be able to:

1. Design the experiment for the given problem using HTML, Javascript and CSS.
2. Develop the solution for the given real-world problem using jQuery, Ajax and PHP.
3. Analyze the results and produce substantial written documentation

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

Continuous Internal Evaluation (CIE): CIE marks for the practical course are 50 Marks. The split-up of CIE marks for record/ journal and test are in the ratio 60:40.

● Each experiment is to be evaluated for conduction with an observation sheet and record write-up. Rubrics for the evaluation of the journal/write-up for hardware/software experiments are designed by the faculty who is handling the laboratory session and are made known to students at the beginning of the practical session.
● Record should contain all the specified experiments in the syllabus and each experiment write-up will be evaluated for 10 marks.
● Total marks scored by the students are scaled down to 30 marks (60% of maximum marks).
● Weightage to be given for neatness and submission of record/write-up on time.
● Department shall conduct a test of 100 marks after the completion of all the experiments listed in the syllabus.
● In a test, test write-up, conduction of experiment, acceptable result, and procedural knowledge will carry a weightage of 60% and the rest 40% for viva-voce.
● The suitable rubrics can be designed to evaluate each student's performance and learning ability.

CSE/CEC

● The marks scored shall be scaled down to 20 marks (40% of the maximum marks). The Sum of scaled-down marks scored in the report write-up/journal and marks of a test is the total CIE marks scored by the student.

Semester End Evaluation (SEE):

● SEE marks for the practical course are 50 Marks.

● SEE shall be conducted jointly by the two examiners of the same institute; examiners are appointed by the Head of the Institute.

● The examination schedule and names of examiners are informed to the university before the conduction of the examination. These practical examinations are to be conducted between the schedule mentioned in the academic calendar of the University.

CSE/ CEC

Introduction

What is Web Development?

Web development refers to building, creating, and maintaining websites. It includes aspects such as web design, web publishing, web programming, and database management. It is the creation of an application that works over the internet, i.e., websites.

There are two major areas: **Frontend** and **Backend** which forms the backbone of web development each plays a crucial role in creating seamless, functional web experiences.

Frontend Development

In this module, we explore the core technologies that run in the user's browser—the client side—including how web pages are structured, styled, and made interactive, building everything users see and interact with.

- **HTML (HyperText Markup Language):** HTML is the language used to create the basic structure and content of web pages. It uses elements, tags, and attributes to organize text, images, and links.
- **CSS (Cascading Style Sheets):** CSS is used to style the HTML content. It controls colors, fonts, layouts, and how the page looks on different devices. More importantly, CSS enables you to do this independent of the HTML that makes up each web page.
- **JS (JavaScript):** JavaScript adds life to web pages by making them interactive. It handles things like buttons, animations, and form checks.

Backend Development:

In this module, we will explore the technologies that work behind the scenes on the server to handle data, run the website, and store information.

Server-Side Programming Languages

In Backend Development, Server-side programming languages are used to write code that runs on the server, not in the user's browser. This server-side scripting handles tasks like processing data, managing databases, and controlling how the website works behind the scenes

Below are some popular languages used to build the back end of web applications:

- **JavaScript/Node.js:** JavaScript is a popular programming language mainly used to add interactivity on the client side (in browsers). With Node.js, JavaScript can also run on the server side. Node.js is an open-source environment that allows JavaScript to build fast, scalable back-end services like APIs. Many big companies like PayPal, Uber, and Netflix use Node.js for their server-side code.
- **PHP:** PHP is a server-side scripting language designed specifically for web development. Since PHP code executed on the server-side, so it is called a server-side scripting language.
- **Python:** Python is a programming language that lets you work quickly and integrate systems more efficiently.
- **Ruby:** An object-oriented programming language designed to be simple and natural to use. Ruby helps developers write clean and readable code.
- **Java:** Java is one of the most popular and widely used programming languages and platforms. It is highly scalable. Java components are easily available.
- **Golang(Go):** Golang is a procedural and statically typed programming language having the syntax similar to C programming language. Sometimes it is termed as Go Programming Language.
- **C#:** A modern, object-oriented language often used to build web applications on Microsoft platforms.

CSE/CEC

Databases

A database is where a website's data like user's data, product's data are stored and organized. It is part of the backend (server side) that manages and keeps this information safe. Websites use databases to save and access information like user details, content, and transactions. Some databases organize data in tables (called relational databases, like MySQL), while others store data in flexible formats (called NoSQL databases, like MongoDB).

APIs and Data Exchange Formats

During Website development, different software components and web applications constantly need to communicate and share information. For instance, the frontend of your web application (running in the user's browser) needs to get data from the backend (running on a server), or your application might need to fetch information from a third-party service like a weather provider or a payment gateway. This communication is made possible through Application Programming Interfaces (APIs) and standardized Data Formats.

Data Exchange format for API Communication: When applications communicate via APIs, they need a common, structured way to represent the data being exchanged. This is where data formats come in.

Below are two common data formats used extensively in web development for API communication:

- **JSON:** JSON or JavaScript Object Notation is a format for structuring data.
- **XML:** Extensible Markup Language (XML) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable.

Protocols and the Internet

Web technology works over the internet using a set of rules called **protocols**, which define how data is transmitted between computers. The most important ones include:

- **HTTP (HyperText Transfer Protocol)** – The foundation of data communication on the web. It transfers web pages from the server to the user's browser.
- **HTTPS (HTTP Secure)** – A secure version of HTTP that uses encryption (SSL/TLS) to protect data during transfer.
- **FTP (File Transfer Protocol)** – Used to upload or download files between a client and server.
- **SMTP (Simple Mail Transfer Protocol)** – Handles the sending of emails.
- **IMAP/POP3** – Protocols used for receiving emails.
- **DNS (Domain Name System)** – Converts human-friendly website names (like www.example.com) into IP addresses that computers use to identify each other.
- **IP (Internet Protocol)** – Defines how data is addressed and routed across networks.

How It All Works Together

When a user types a website address in the browser, DNS finds the server's IP address. The browser then sends a request via HTTP/HTTPS to the server, the server processes it (backend), retrieves data from databases if needed, and sends the result back. The frontend then displays it in a user-friendly way, sometimes fetching additional data through APIs.

CSE/CEC

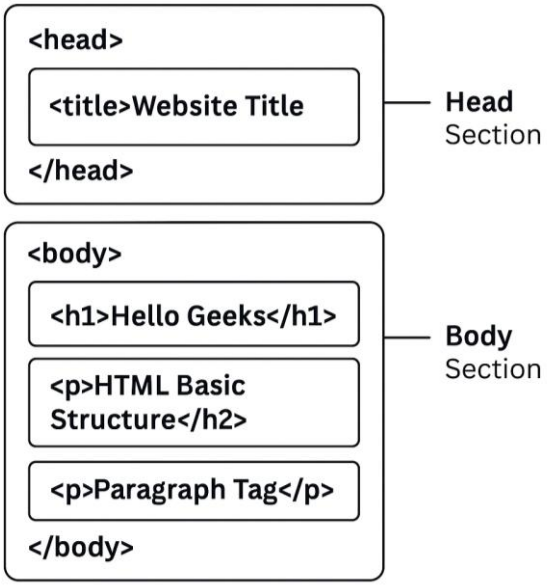
HTML

HTML stands for **HyperText Markup Language**. It is the standard language used to create and structure content on the web.

- It defines the layout of a webpage using elements and tags, allowing for the display of text, images, links, and multimedia content.
- As the foundation of nearly all websites, HTML is used in over 95% of all web pages today, making it an essential part of modern web development.

HTML sets up the basic structure of a website, and then CSS and JavaScript add style and interactivity to make it look and function better.

HTML – Structure of Web Pages



The diagram shows the **basic structure of an HTML document** and how its parts work together:

1. – Tells the browser that the page is written in HTML5.
2. **...** – The root element that contains the entire HTML content.
3. **...** – Holds metadata about the webpage, such as the title, character set, styles, and linked scripts.
   - – Sets the page title shown in the browser tab.
4. **...** – Contains the visible content of the webpage (text, images, links, tables, etc.).
5. HTML tags are written in pairs: an **opening tag** (e.g.,) and a **closing tag** (e.g.,), with the content in between.

In short, this structure tells the browser **what the page is** (DOCTYPE), **describes it** (head), and **shows the content** (body).

HTML Syntax – Elements and Attributes

1. **HTML Document Structure**
   - HTML documents are made up of **text content** and **HTML elements**.
2. **HTML Elements vs Tags**
   - An **HTML element** includes:
     - The **opening tag** (element name in angle brackets)
     - The **content** inside the tag
     - The **closing tag**
   - Example:
   - Hello, I am a paragraph

Here:

- → Opening tag
- Hello, I am a paragraph → Content
- → Closing tag

3. **Tags** are special keywords in HTML enclosed within angle brackets < >. They tell the browser how to display content
   - A **tag** is the element name inside **angle brackets** (< >).
   - **Opening tag**: <elementName>
   - **Closing tag**: </elementName> (includes /).
4. **Attributes**
   - Attributes give extra information about an element.
   - They are written inside the **opening tag**.
   - Example:
   - Hello, I am a paragraph
     - style="font-size:40px" is an attribute.

Explanation:

- <!DOCTYPE html> → Declares HTML version.
- <html> → Root element.
- <head> → Contains metadata (title, styles, scripts).
- <title> → Sets the page title in the browser tab.
- <body> → Contains the visible content.
- <h1> → Heading tag.
- → Paragraph tag.
- style="..." → Attribute to add inline CSS styling.

CSE/CEC

Types Of Tags

1. Structural Tags

These define the **basic structure** of an HTML document.

- <html> → The root element that contains all HTML code.
- <head> → Contains metadata (title, CSS links, scripts) that is not directly visible on the webpage.
- <body> → Contains the actual visible content of the webpage.

2. Text Formatting

These tags control how **text appears**.

- <h1> to <h6> → Headings, with <h1> being the largest and <h6> the smallest.
- <p> → Paragraph tag, used for blocks of text.
-
 → Line break (moves content to the next line without starting a new paragraph).
- → Makes text **bold** and gives it semantic importance.
- → Emphasizes text, usually shown in *italic*.

3. Links and Media

Used to add **navigation** and **multimedia**.

- <a href="URL"> → Creates a hyperlink to another page or location.
- <img src="image.jpg"> → Displays an image.
- <video> → Embeds a video (can have controls like play/pause).
- <audio> → Embeds an audio file (with play/pause controls).

4. Lists

Used to display **items in order or without order**.

- <ul> → Unordered list (bullets).
- <ol> → Ordered list (numbers or letters).
- <li> → List item (used inside <ul> or <ol>).

5. Tables

Used for displaying **tabular data**.

- <table> → Defines the table.
- <tr> → Table row.
- <td> → Table cell (normal data).
- <th> → Table header cell (bold and centered by default).

6. Forms

CSE/ CEC

Used to collect **user input**.

- <form> → Defines a form container.
- <input> → Single-line input field (text, number, password, etc.).
- <textarea> → Multi-line input box.
- <select> → Drop-down menu.
- <option> → Item inside a <select> dropdown.
- <button> → Clickable button (can submit forms or trigger actions).

1. **Develop the HTML page named as "Myfirstwebpage.html". Add the following tags with relevant content.**
   1. Set the title of the page as "My First Web Page"
   2. Within the body use the following tags:
   a) Moving text = "Basic HTML Tags" b) Different heading tags (h1 to h6)
   c) Paragraph d) Horizontal line
   e) Line Break f) Block Quote g) Pre tag h) Different Logical Style (, , etc.)

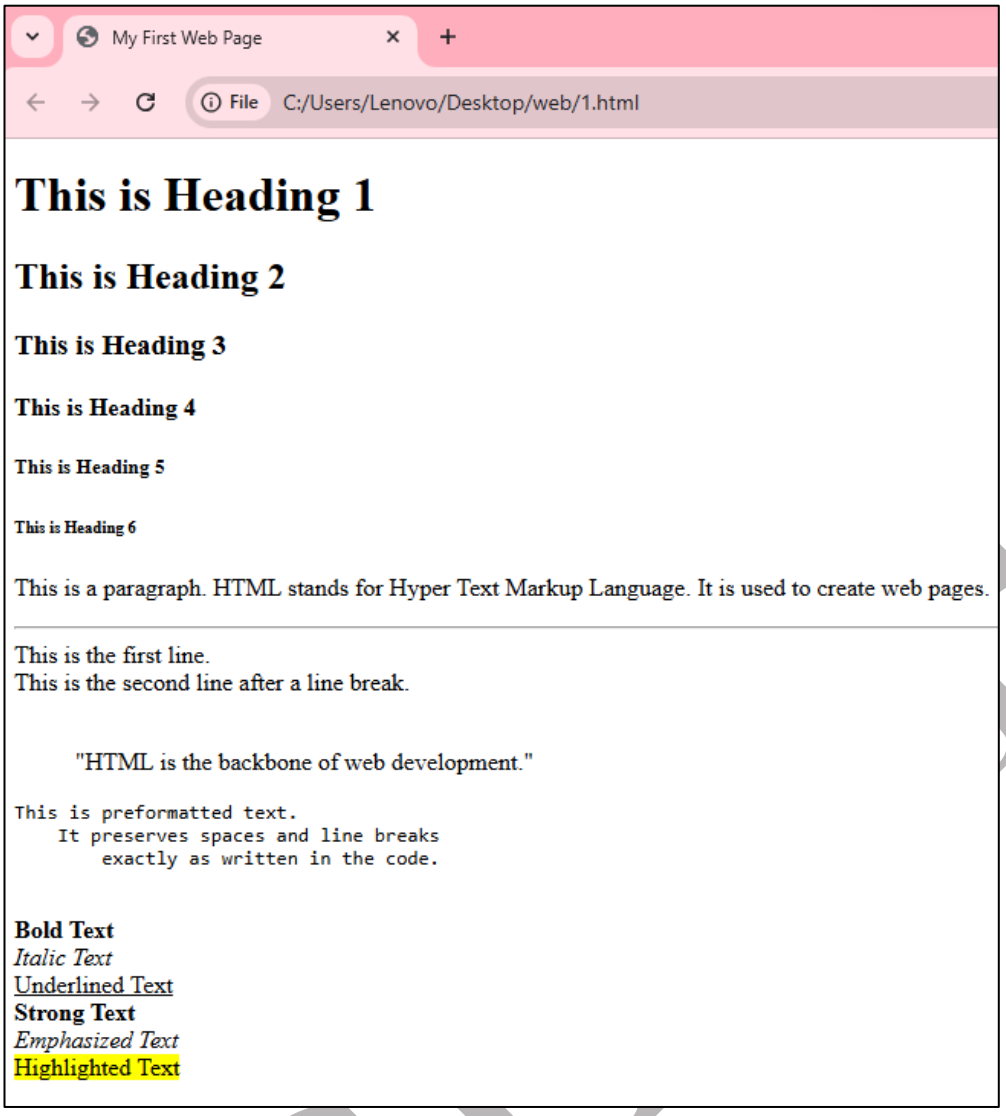
```
<!DOCTYPE html>
<html>
<head>
   <title>My First Web Page</title>
</head>
<body>
   <!-- Moving Text -->
   <marquee>Basic HTML Tags</marquee>
   <!-- Headings -->
   <h1>This is Heading 1</h1>
   <h2>This is Heading 2</h2>
   <h3>This is Heading 3</h3>
   <h4>This is Heading 4</h4>
   <h5>This is Heading 5</h5>
   <h6>This is Heading 6</h6>
   <!-- Paragraph -->
   <p>  This is a paragraph. HTML stands for Hyper Text Markup Language. It is used to create web pages
</p>
```

CSE/ CEC

```
    <!-- Horizontal Line -->
    <hr>
    <!-- Line Break -->
    This is the first line.<br>
    This is the second line after a line break.<br><br>
    <!-- Block Quote -->
    <blockquote>
        "HTML is the backbone of web development."
    </blockquote>
    <!-- Preformatted Text -->
    <pre>
This is preformatted text.
    It preserves spaces and line breaks
        exactly as written in the code.
    </pre>
    <!-- Logical Styles -->
    <p>
        <b>Bold Text</b><br>
        <i>Italic Text</i><br>
        <u>Underlined Text</u><br>
        <strong>Strong Text</strong><br>
        <em>Emphasized Text</em><br>
        <mark>Highlighted Text</mark>
    </p>
</body>
</html>
```

CSE/ CEC

OUTPUT



HTML Tables

Introduction to HTML Tables

An HTML table is used to display data in a structured format using rows and columns.
It is created using the <table> tag, which contains <tr> for rows and <td> for table data cells.

Basic Table Tags

Tag	Purpose
<table>	Defines the table.
<tr>	Defines a row in the table.
<th>	Defines a header cell (bold and centered by default).
<td>	Defines a data cell.

Table Sections

- **<thead>** – Represents the header section of the table.
- **<tbody>** – Represents the main content (body) of the table.
- **<tfoot>** – Represents the footer section of the table.

Attributes for Table Cells

- **colspan="n"** – Merges multiple columns into one cell.
- **rowspan="n"** – Merges multiple rows into one cell.

Example:

```
<td colspan="2">Merged Cell</td>
<td rowspan="3">Merged Vertically</td>
```

Table Styling

- **Borders**: border="1" (basic HTML) or CSS border: 1px solid black;
- **Cell Spacing**: cellspacing – space between cells.
- **Cell Padding**: cellpadding – space inside each cell.
- **Background Color**: Using bgcolor (HTML) or background-color (CSS).
- **Row Color**: Can be set using CSS selectors like tr:nth-child(even).

Highlighting Specific Cells

- **Lab hours** – Can be given a specific background color using a CSS class.
- **Elective hours** – Can have a different color for easy identification.

7. Table Header and Footer

- Header (<thead>) usually contains the column names.
- Footer (<tfoot>) may contain a summary or note about the table.
- These sections help in better readability and organization.

2. Develop the HTML page named as "Table.html" to display your class time table.
a) Provide the title as Time Table with table header and table footer, row-span and col-span etc.
b) Provide various colour options to the cells (Highlight the lab hours and elective hours with different colours.) c) Provide colour options for rows.

```
<!DOCTYPE html>
<html>
<head>
  <title>Time Table</title>
  <style>
```

CSE/CEC

```
        table {
            border-collapse: collapse;
            width: 100%;
            text-align: center;
        }
        th, td {
            border: 1px solid black;
            padding: 8px;
        }
        thead {
            background-color: #ffcccb;
        }
        tfoot {
            background-color: #d1c4e9;
        }
        .lab {
            background-color: #a5d6a7;
        }
        .elective {
            background-color: #ffeb3b;
        }
        tr:nth-child(even) {
            background-color: #f2f2f2;
        }
        tr:nth-child(odd) {
            background-color: #e3f2fd;
        }
    </style>
</head>
<body>
```

CSE/ CEC

```
<h2 style="text-align:center;">Class Time Table</h2>


<table>
  <thead>
    <tr>
      <th>Day</th>
      <th>9:00 - 10:00</th>
      <th>10:00 - 11:00</th>
      <th>11:00 - 12:00</th>
      <th>12:00 - 1:00</th>
      <th>1:00 - 2:00</th>
      <th>2:00 - 3:00</th>
      <th>3:00 - 4:00</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Monday</td>
      <td>Math</td>
      <td>English</td>
      <td rowspan="2" class="lab">Physics Lab</td>
      <td>History</td>
      <td>Lunch</td>
      <td class="elective">Elective - Art</td>
      <td>Sports</td>
    </tr>
    <tr>
      <td>Tuesday</td>
      <td>Chemistry</td>
      <td>Biology</td>
      <td>Geography</td>
```

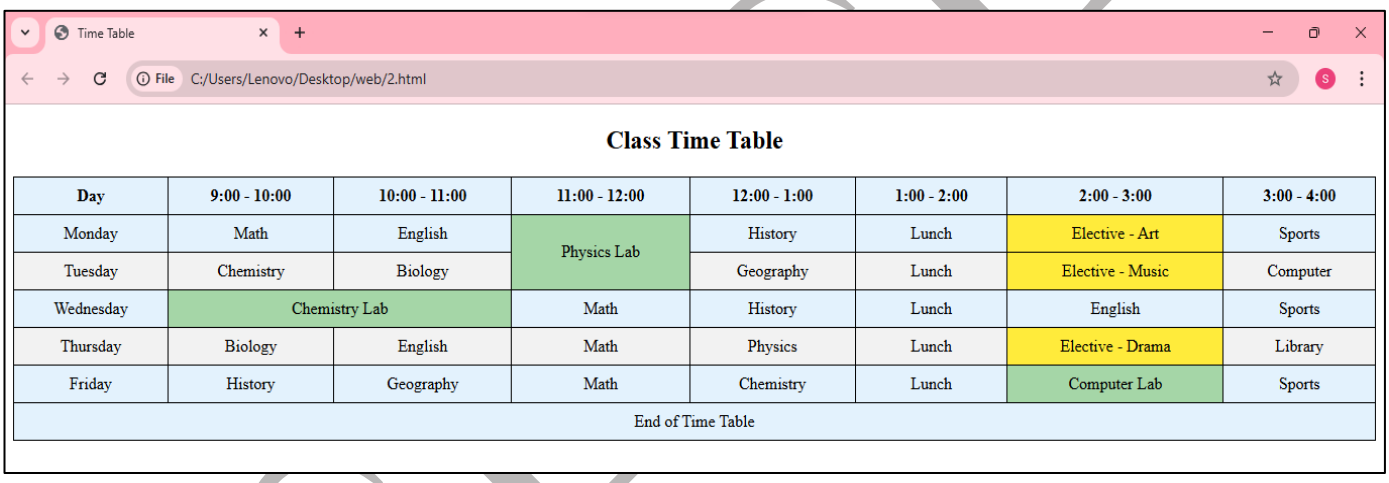
CSE/ CEC

```
        <td>Lunch</td>
        <td class="elective">Elective - Music</td>
        <td>Computer</td>
    </tr>
    <tr>
        <td>Wednesday</td>
        <td colspan="2" class="lab">Chemistry Lab</td>
        <td>Math</td>
        <td>History</td>
        <td>Lunch</td>
        <td>English</td>
        <td>Sports</td>
    </tr>
    <tr>
        <td>Thursday</td>
        <td>Biology</td>
        <td>English</td>
        <td>Math</td>
        <td>Physics</td>
        <td>Lunch</td>
        <td class="elective">Elective - Drama</td>
        <td>Library</td>
    </tr>
    <tr>
        <td>Friday</td>
        <td>History</td>
        <td>Geography</td>
        <td>Math</td>
        <td>Chemistry</td>
        <td>Lunch</td>
        <td class="lab">Computer Lab</td>
```

CSE/ CEC

```
            <td>Sports</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <td colspan="8">End of Time Table</td>
        </tr>
    </tfoot>
</table>
</body>
</html>
```

OUTPUT



Class Time Table

Day	9:00 - 10:00	10:00 - 11:00	11:00 - 12:00	12:00 - 1:00	1:00 - 2:00	2:00 - 3:00	3:00 - 4:00
Monday	Math	English	Physics Lab	History	Lunch	Elective - Art	Sports
Tuesday	Chemistry	Biology		Geography	Lunch	Elective - Music	Computer
Wednesday	Chemistry Lab		Math	History	Lunch	English	Sports
Thursday	Biology	English	Math	Physics	Lunch	Elective - Drama	Library
Friday	History	Geography	Math	Chemistry	Lunch	Computer Lab	Sports
End of Time Table							

CSS – Styling Web Pages`

What is CSS?

CSS (Cascading Style Sheets) is used to control the look and layout of HTML elements.
It separates **content (HTML)** from **presentation (design)**.

Types of CSS

Type	Where It's Written	Usage
Inline CSS	Inside the HTML element using the style attribute.	For quick, one-time styling.
Internal CSS	Inside <style> tags in the <head> of HTML.	For styling one HTML file.
External CSS	In a separate .css file, linked using <link>.	For applying same styles to multiple HTML files.

Syntax of CSS

```
selector {
   property: value;
}
```

- **Selector** – HTML element you want to style (h2, p, div, etc.)
- **Property** – What you want to change (color, font-size, etc.)
- **Value** – Setting for that property.

Example:

```
p {
   color: blue;
   font-size: 16px;
}
```

External CSS

- Written in a separate file with **.css** extension.
- Linked in HTML using:

```
<link rel="stylesheet" type="text/css" href="style.css">
```

Explanation of CSS Selectors and Significance

Selector	Description	Significance
h2 (Element selector)	Targets all <h2> tags.	Applies uniform styles to all <h2> headings.
div.container (Class selector)	Targets elements with class="container".	Style reusable containers separately.
span#highlight (ID selector)	Targets the with id="highlight".	Style a unique element specifically.
time[datetime] (Attribute selector)	Targets <time> tags having the datetime attribute.	Styles semantic time elements with date/time info.
div.container img (Descendant selector)	Targets <img> inside div.container only.	Styles images only in a specific section.
a:hover (Pseudo-class selector)	Targets links on mouse hover.	Adds interactivity with visual feedback.
h2, h3 (Group selector)	Targets both <h2> and <h3> tags.	Avoids repetition; applies common styles.

3. Develop an external style sheet named as "style.css" and provide different styles for h2, h3, hr, p, div, span

Step 1: Create the HTML File (index.html)

1. Open any text editor (Notepad, Notepad++, VS Code, Sublime Text, etc.).
2. Type the HTML code given below:
3. Save the file as **index.html** in a folder.

CSE/CEC

index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>External CSS Example</title>
   <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
<h2>Welcome to External CSS</h2>
<h3>Example of Styling Elements</h3>
<hr>
<div>
   <p>
      This is a paragraph inside a div.
      External CSS allows you to style multiple pages easily.
      The word <span>important</span> is styled using the span selector.
   </p>
</div>
<p>
   This is another paragraph outside the div. The styles for paragraphs are applied from the external CSS file.
</p>
</body>
</html>
```

Step 2: Create the CSS File (style.css)

1. In the same folder where you saved index.html, create a new file.
2. Type the CSS code.
3. Save the file as style.css.
4. Ensure it is saved in the same folder as index.html.

style.css

```
/* h2 heading style */
h2 {
   color: darkblue;
```

CSE/CEC

```
    text-align: center;
    font-family: Arial, sans-serif;
    font-size: 28px;
}
/* h3 heading style */
h3 {
    color: darkgreen;
    text-decoration: underline;
    font-family: Verdana, sans-serif;
    font-size: 22px;
}
/* Horizontal line style */
hr {
    border: 2px solid gray;
    margin: 20px 0;
}
/* Paragraph style */
p {
    color: #555;
    font-size: 16px;
    line-height: 1.5;
    text-align: justify;
}
/* Division style */
div {
    background-color: #f0f8ff;
    padding: 15px;
    border: 1px solid #ccc;
    margin-bottom: 20px;
}
/* Span style */
```

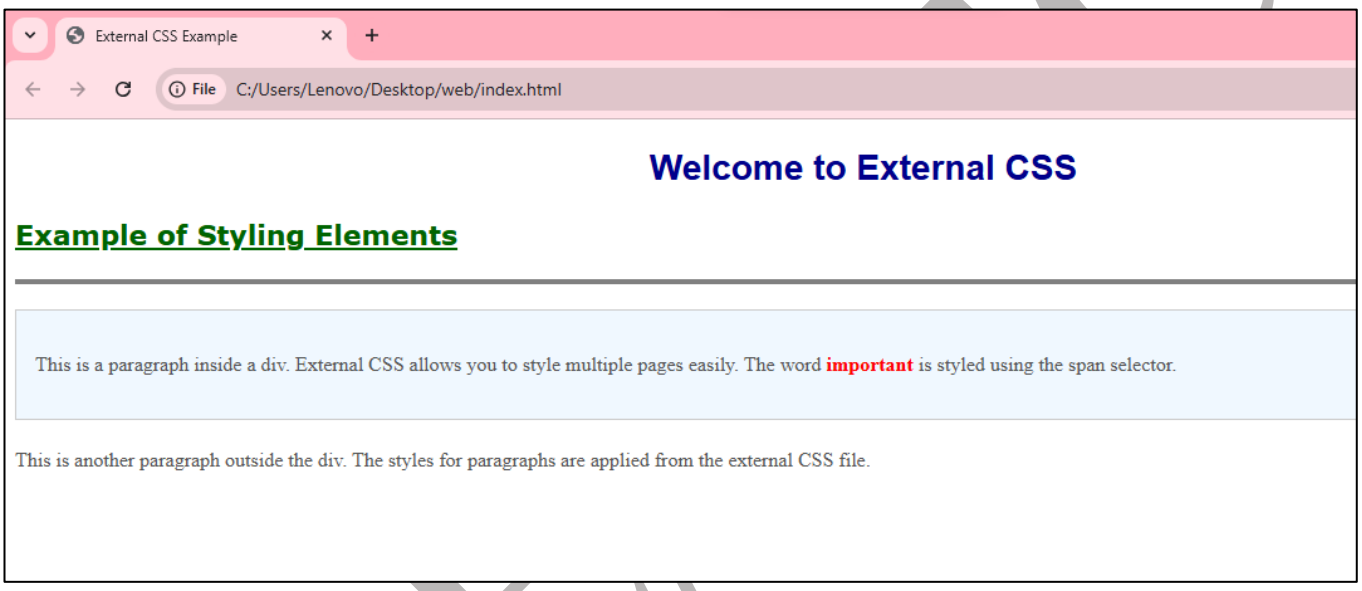
CSE/ CEC

```
span {
   color: red;
   font-weight: bold;
}
```

Step 3: Run the Program

1. Go to the folder where both files are saved.
2. Double-click **index.html**.
3. It will open in your default web browser with the applied CSS styles.

OUTPUT



HTML Forms

Purpose of Forms

- HTML forms are used to **collect user input**.
- Data entered into a form can be **sent to a server** for processing (e.g., saving to a database, sending an email).
- Common uses: login forms, registration forms, search bars, feedback forms, payment details.

Basic Structure of a Form

An HTML form is basically made up of three main parts:

1. Form container → <form> tag with its attributes.
2. Form controls → Different input fields like text boxes, radio buttons, checkboxes, etc.
3. Submission controls → Buttons to submit or reset the form.

Form Container – `<form>` Tag

The `<form>` tag wraps all the form elements.

Syntax:

```
<form action="server_page.php" method="post">
    <!-- form elements here -->
</form>
```

Important attributes:

Attribute	Purpose
`action`	The URL or file where the form data will be sent for processing (server-side script like PHP, ASP, etc.).
`method`	The HTTP method for sending data: `GET` or `POST`.
`enctype`	Defines how form data is encoded before sending. Needed for file uploads (`multipart/form-data`).
`name`	Gives a name to the form (optional, for scripts).
`target`	Specifies where to display the response (`_self`, `_blank`, etc.).

Form input Elements

Element	Purpose	Example
Text Box <input type="text">>	Single-line text entry	Name
Password Field <input type="password">>	Hidden characters for security	Password
Email Field <input type="email">>	Validates email format	Email ID
Number Field <input type="number">>	Accepts only numbers	Age
Date Picker <input type="date">>	Select a date	Date of Birth
Radio Buttons <input type="radio">>	Choose one option from many	Gender
Checkboxes <input type="checkbox">>	Choose multiple options	Hobbies
Dropdown <select>>	Select one from a list	Country
Textarea <textarea>>	Multi-line text	Address
File Upload <input type="file">>	Upload a file	Profile picture
Buttons <input type="submit">> / <input type="reset">>	Send or clear form	Submit/Reset

CSE/CEC

Attributes of Form Elements

- **name** → The name of the field (used for identifying data on the server).
- **value** → Default value of the field.
- **placeholder** → Hint text inside the field.
- **required** → Makes a field mandatory.
- **checked** → Pre-selects a checkbox or radio button.
- **disabled** → Makes an input uneditable.
- **maxlength** → Limits number of characters.

Example:

```
<input type="text" name="username" placeholder="Enter your name" required>
```

Grouping Related Fields

- Use <fieldset> and <legend> to group related form elements with a caption.

```
<fieldset>
  <legend>Personal Information</legend>
  Name: <input type="text"><br>
  Age: <input type="number">
</fieldset>
```

Styling Forms

- CSS can be applied to improve readability and user experience:
  - **Padding & margins** → Space between inputs.
  - **Borders & backgrounds** → Highlight sections.
  - **Hover effects** → Improve button interaction.
  - **Responsive design** → Ensure form looks good on all devices.

Form Submission

When you click **Submit**:

1. Browser collects all form data.
2. Sends data to the URL in action.
3. The server processes and may respond with a success message or redirect.

CSE/ CEC

4. Develop HTML page named as "registration.html" having variety of HTML input elements with background colors, table for alignment & provide font colors & size using CSS styles.

registration.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Registration Form</title>
   <style>
      /* Page background */
      body {
         background-color: #f2f2f2;
         font-family: Arial, sans-serif;
      }

      /* Heading style */
      h2 {
         color: darkblue;
         text-align: center;
         font-size: 28px;
      }
      /* Table styling */
      table {
         background-color: white;
         margin: auto;
         border: 2px solid #333;
         padding: 15px;
         border-radius: 10px;
      }
      td {
         padding: 8px;
         font-size: 16px;
         color: #333;
      }
      /* Input fields styling */
      input, select, textarea {
         padding: 5px;
         font-size: 14px;
         border: 1px solid #aaa;
         border-radius: 4px;
      }
      /* Submit and Reset button style */
      input[type="submit"], input[type="reset"] {
         background-color: darkblue;
         color: white;
         font-size: 16px;
         padding: 6px 12px;
         border: none;
         border-radius: 5px;
         cursor: pointer;
      }
```

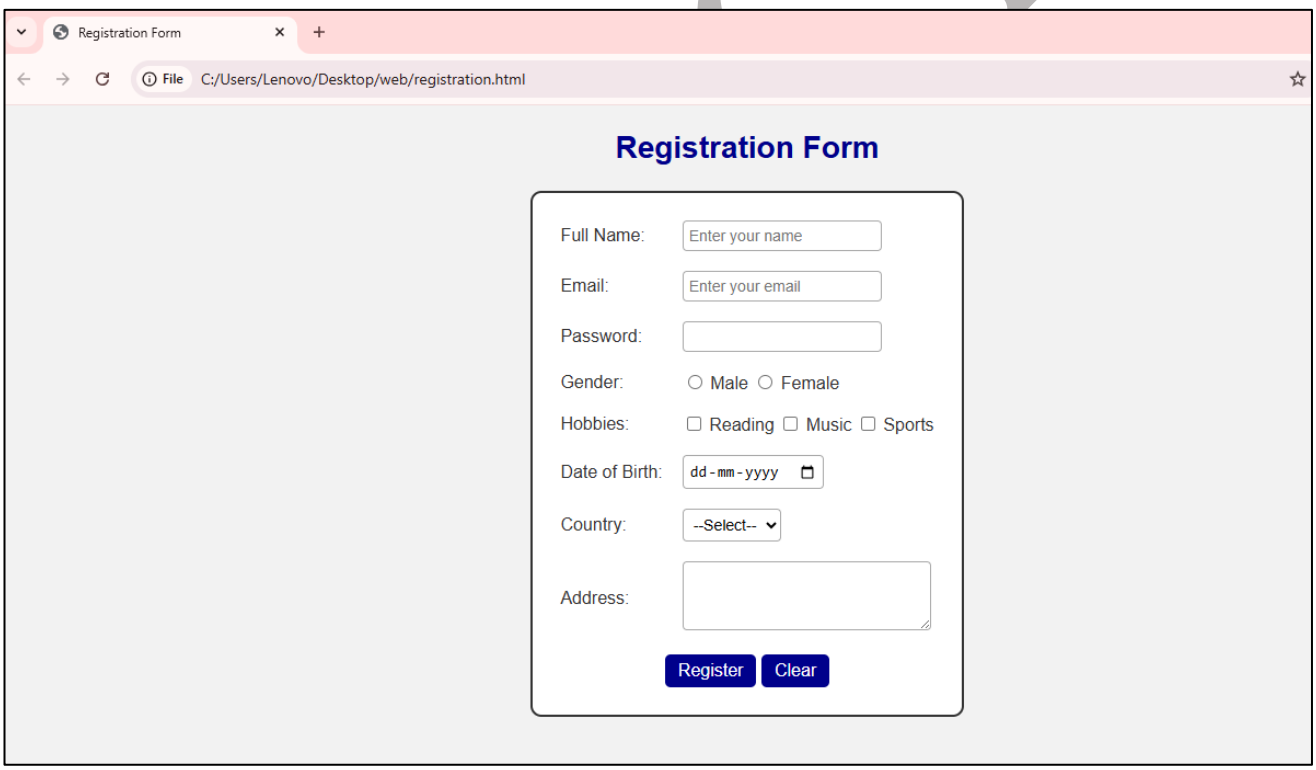
CSE/ CEC

```
        input[type="submit"]:hover, input[type="reset"]:hover {
            background-color: navy;
        }
    </style>
</head>
<body>

    <h2>Registration Form</h2>

    <form action="#" method="post">
        <table>
            <tr>
                <td><label for="name">Full Name:</label></td>
                <td><input type="text" id="name" name="fullname" placeholder="Enter your name"
required></td>
            </tr>
            <tr>
                <td><label for="email">Email:</label></td>
                <td><input type="email" id="email" name="email" placeholder="Enter your email"></td>
            </tr>
            <tr>
                <td><label for="password">Password:</label></td>
                <td><input type="password" id="password" name="password"></td>
            </tr>
            <tr>
                <td>Gender:</td>
                <td>
                    <input type="radio" name="gender" value="Male"> Male
                    <input type="radio" name="gender" value="Female"> Female
                </td>
            </tr>
            <tr>
                <td>Hobbies:</td>
                <td>
                    <input type="checkbox" name="hobby" value="Reading"> Reading
                    <input type="checkbox" name="hobby" value="Music"> Music
                    <input type="checkbox" name="hobby" value="Sports"> Sports
                </td>
            </tr>
            <tr>
                <td><label for="dob">Date of Birth:</label></td>
                <td><input type="date" id="dob" name="dob"></td>
            </tr>
            <tr>
                <td><label for="country">Country:</label></td>
                <td>
                    <select id="country" name="country">
                        <option value="">--Select--</option>
                        <option>India</option>
                        <option>USA</option>
                        <option>UK</option>
                    </select>
```

CSE/CEC

```
            </td>
        </tr>
        <tr>
            <td><label for="address">Address:</label></td>
            <td><textarea id="address" name="address" rows="3" cols="25"></textarea></td>
        </tr>
        <tr>
            <td colspan="2" style="text-align:center;">
                <input type="submit" value="Register">
                <input type="reset" value="Clear">
            </td>
        </tr>
    </table>
  </form>

</body>
</html>
```

OUTPUT



HTML Semantic Elements

What are Semantic Elements?

In HTML, **semantic elements** are tags that describe the meaning or purpose of the content they contain, rather than just its appearance.
This makes web pages:

- **Easier to read** (for humans and developers)
- **More accessible** (for screen readers)
- **Better for Search Engine Optimization** (search engines understand content meaning)

Example:

- <header> clearly indicates the top section of a page.
- <article> indicates an independent block of content.
- <footer> indicates the bottom section.

Common HTML Semantic Elements

Semantic Element	Description
<header>	Represents the top section of a webpage or a section, usually containing headings, navigation menus, or logos.
<nav>	Defines a navigation section with links to other pages or parts of the same page.
<section>	Represents a thematic grouping of content, typically with its own heading.
<article>	Represents independent, self-contained content (like a blog post, news article, or product listing).
<aside>	Represents content related to the main content (e.g., sidebars, ads, additional info).
<figure>	Used to group media (image, diagram, illustration) along with a <figcaption>.
<figcaption>	Provides a caption or description for the <figure> element.
<footer>	Represents the bottom section of a page or section, often containing contact info, copyright, or links.
<main>	Represents the main content area of the webpage.
<time>	Represents a specific time or date.

Benefits of Semantic HTML

- **Improved Accessibility** – Screen readers can interpret the document structure better.
- **Better SEO** – Search engines use semantic tags to rank pages.
- **Cleaner Code** – Improves readability and maintainability.
- **Future-Proofing** – Standardized HTML5 structure.

CSE/CEC

Example of Semantic Layout

A typical HTML5 page structure:

```
<header>…</header>
<nav>…</nav>
<main>
  <section>
    <article>…</article>
  </section>
  <aside>…</aside>
</main>
<footer>…</footer>
```

5. **Develop HTML page named as "newpaper.html" having variety of HTML semantic elements with background colors, text-colors & size for figure, table, aside, section, article, header, footer… etc.**

newspaper.html

```
<!DOCTYPE html>
<html>
<head>
   <title>My Newspaper</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         margin: 0;
         background-color: #f2f2f2;
      }

      header {
         background-color: #004080;
         color: white;
         text-align: center;
         padding: 20px;
         font-size: 28px;
```

CSE/ CEC

```
}

nav {
    background-color: #333;
    padding: 10px;
    text-align: center;
}

nav a {
    color: white;
    text-decoration: none;
    margin: 0 15px;
    font-size: 16px;
}
section {
    background-color: white;
    padding: 15px;
    margin: 15px;
    border-radius: 8px;
}
article {
    background-color: #e6f0ff;
    padding: 10px;
    margin-bottom: 10px;
}
aside {
    background-color: #fffae6;
    padding: 10px;
    margin: 15px;
    border: 1px solid #ccc;
}
```

CSE/ CEC

```
    footer {
      background-color: #004080;
      color: white;
      text-align: center;
      padding: 15px;
      font-size: 14px;
    }
    figure {
      text-align: center;
      margin: 10px;
    }
    figure img {
      width: 200px;
      height: auto;
      border-radius: 5px;
    }
    figcaption {
      font-size: 14px;
      color: #555;
    }
    table {
      border-collapse: collapse;
      width: 100%;
      margin-top: 10px;
    }
    table, th, td {
      border: 1px solid #333;
      padding: 8px;
      text-align: center;
}
    th {
```

CSE/ CEC

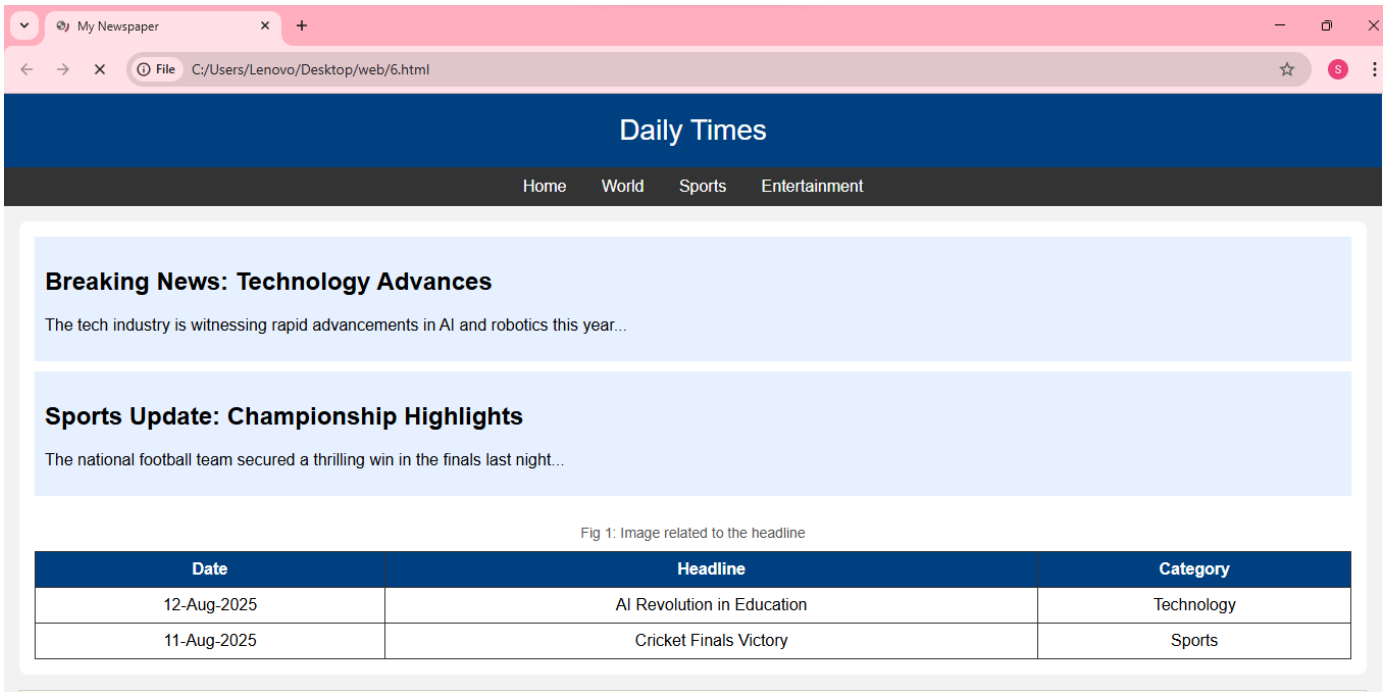
```
            background-color: #004080;
            color: white;
        }
    </style>
</head>
<body>
    <header>
        Daily Times
    </header>
    <nav>
        <a href="#">Home</a>
        <a href="#">World</a>
        <a href="#">Sports</a>
        <a href="#">Entertainment</a>
    </nav>
    <section>
        <article>
            <h2>Breaking News: Technology Advances</h2>
            <p>The tech industry is witnessing rapid advancements in AI and robotics this year...</p>
        </article>
        <article>
            <h2>Sports Update: Championship Highlights</h2>
            <p>The national football team secured a thrilling win in the finals last night...</p>
        </article>
        <figure>
            <img src="https://via.placeholder.com/200" alt="News Image">
            <figcaption>Fig 1: Image related to the headline</figcaption>
        </figure>
        <table>
            <tr>
                <th>Date</th>
```

CSE/ CEC

```
            <th>Headline</th>
            <th>Category</th>
        </tr>
        <tr>
            <td>12-Aug-2025</td>
            <td>AI Revolution in Education</td>
            <td>Technology</td>
        </tr>
        <tr>
            <td>11-Aug-2025</td>
            <td>Cricket Finals Victory</td>
            <td>Sports</td>
        </tr>
    </table>
  </section>

  <aside>
    <h3>Advertisements</h3>
    <p>Buy one get one free on all electronics! Offer valid till 15th August.</p>
  </aside>

  <footer>
    &copy; 2025 Daily Times. All Rights Reserved.
  </footer>

</body>
</html>
```

CSE/ CEC

OUTPUT



JavaScript

Introduction

- **JavaScript** is a lightweight, interpreted, object-oriented scripting language.
- It is mainly used for making **web pages interactive**.
- It runs in the browser (client-side) but can also run on the server (Node.js).
- Developed by **Brendan Eich** in 1995 at Netscape.

Features

- **Lightweight & Fast** – Runs in browsers without additional tools.
- **Dynamic** – Variables can store different data types at different times.
- **Object-Based** – Built around objects like document, window.
- **Event-Driven** – Responds to user actions like clicks, hovers, typing.
- **Cross-Platform** – Works on all modern browsers.

JavaScript vs HTML vs CSS

HTML	CSS	JavaScript
Structure of webpage	Styling of webpage	Behavior & interactivity

CSE/C

How to Include JavaScript

1. **Internal JavaScript** – Written inside <script> tag.

```
<script>
  alert("Hello, World!");
</script>
```

2. **External JavaScript** – Written in .js file and linked using:

```
<script src="script.js"></script>
```

3. **Inline JavaScript** – Written directly inside an HTML element's attribute.

```
<button onclick="alert('Clicked!')">Click Me</button>
```

Variables- Declared using:

```
var name = "John";    // Old method
let age = 25;         // Modern method
const pi = 3.14;      // Constant value
```

Data Types

- **Primitive**: string, number, boolean, null, undefined, symbol, bigint
- **Non-Primitive**: objects, arrays, functions

Example:

```
let name = "Alice";   // string
let age = 20;         // number
let isActive = true;  // boolean
```

Operators

- **Arithmetic**: + - * / % ++ --
- **Assignment**: = += -= *= /=
- **Comparison**: == != === !== > < >= <=
- **Logical**: && || !

8. Conditional Statements

```
if (age >= 18) {
   console.log("Adult");
} else {
   console.log("Minor");
}
switch(day) {
   case 1: console.log("Monday"); break;
   default: console.log("Other day");
}
```

CSE/ CEC

Loops

```
for (let i = 0; i < 5; i++) { console.log(i); }
while (x < 10) { x++; }
do { x++; } while (x < 10);
```

Functions

```
function greet(name) {
   return "Hello " + name;
}
console.log(greet("John"));
```

Arrow Function:

```
let greet = (name) => "Hello " + name;
```

Events

- Common events: onclick, onmouseover, onchange, onkeyup

```
<button onclick="alert('Clicked!')">Click Me</button>
```

Document Object Model (DOM)-Used to **access and change HTML elements** dynamically.

```
document.getElementById("demo").innerHTML = "Hello!";
document.querySelector("p").style.color = "red";
```

Built-in Functions

a. **alert()** displays a popup box with a message and an OK button, used to show information or warnings**.**
```
alert("Hello, welcome!");
```
b. **prompt()** displays a dialog box that asks the user for input and returns the entered value as a string or null if cancelled**.**
```
let name = prompt("Enter your name:", "Guest");
```
c. **confirm()** shows a dialog box with OK and Cancel buttons and returns true if OK is clicked, false otherwise.-
```
let result = confirm("Are you sure?");
```
d. **Math.sqrt()** returns the square root of a number and gives NaN for negative numbers.
```
Math.sqrt(16); // 4
```
e. **Math.pow()** returns the base raised to the exponent power.
```
Math.pow(2, 3); // 8
```
f. **Math.random()** returns a random decimal number between 0 (inclusive) and 1 (exclusive) and is often used with Math.floor() to get random integers.
```
Math.random();       // Example: 0.572
Math.floor(Math.random() * 10); // Random number between 0–9
```
g. **parseInt()** converts a string to an integer and ignores the decimal part.
```
parseInt("42");    // 42
parseInt("42.9");  // 42
```

CSE/CEC

h. **parseFloat()** converts a string to a floating-point number and keeps the decimal.

```
parseFloat("42.9"); // 42.9
```

i. Number() converts a value to a number, either integer or float, and returns NaN if conversion fails.

```
Number("42");   // 42
Number("42.9"); // 42.9
```

Advantages

- Increases interactivity.
- Reduces server load.
- Can validate forms before submitting.

Limitations

- Runs on client-side, so **can be disabled** by user.
- Security concerns for sensitive operations.

6. Apply HTML, CSS and JavaScript to design a simple calculator to perform the following operations: sum, product, difference, remainder, quotient, power, square-root and square.

Calculator.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Simple Calculator</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      text-align: center;
    }
    .calculator {
      background-color: white;
      padding: 20px;
      width: 300px;
      margin: 50px auto;
      border-radius: 10px;
      box-shadow: 0px 0px 10px gray;
    }
    input {
      width: 80%;
      padding: 8px;
      margin: 10px 0;
      font-size: 16px;
    }
    button {
      padding: 10px;
      margin: 5px;
      font-size: 14px;
      background-color: #4CAF50;
```

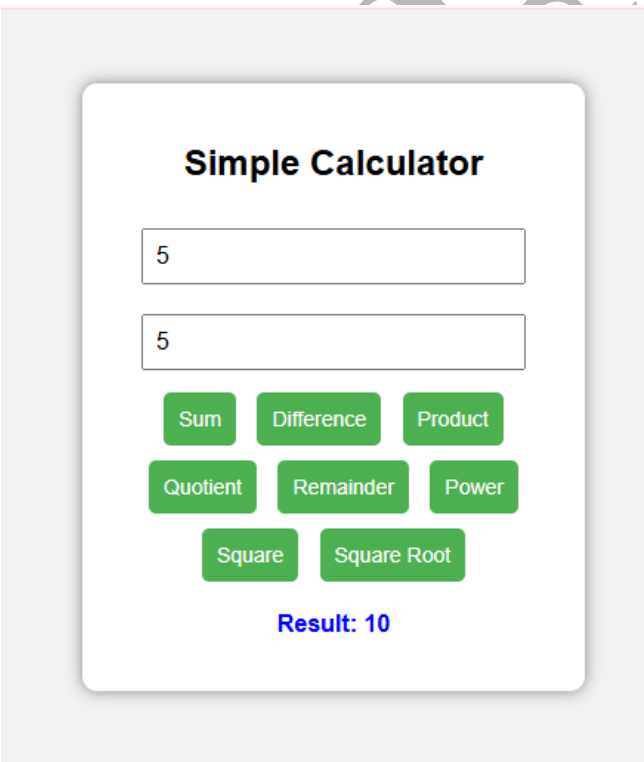
CSE/ CEC

```
            color: white;
            border: none;
            cursor: pointer;
            border-radius: 5px;
        }
        button:hover {
            background-color: #45a049;
        }
        #result {
            margin-top: 15px;
            font-weight: bold;
            color: blue;
        }
    </style>
</head>
<body>

<div class="calculator">
    <h2>Simple Calculator</h2>
    <input type="number" id="num1" placeholder="Enter first number"><br>
    <input type="number" id="num2" placeholder="Enter second number"><br>

    <button onclick="calculate('sum')">Sum</button>
    <button onclick="calculate('difference')">Difference</button>
    <button onclick="calculate('product')">Product</button>
    <button onclick="calculate('quotient')">Quotient</button>
    <button onclick="calculate('remainder')">Remainder</button>
    <button onclick="calculate('power')">Power</button>
    <button onclick="calculate('square')">Square</button>
    <button onclick="calculate('sqrt')">Square Root</button>

    <p id="result">Result: </p>
</div>

<script>
function calculate(operation) {
    let num1 = Number(document.getElementById('num1').value);
    let num2 = Number(document.getElementById('num2').value);
    let res = 0;

    switch(operation) {
        case 'sum':
            res = num1 + num2;
            break;
        case 'difference':
            res = num1 - num2;
            break;
        case 'product':
```

CSE/CEC

```
            res = num1 * num2;
            break;
        case 'quotient':
            res = num2 !== 0 ? num1 / num2 : "Cannot divide by zero";
            break;
        case 'remainder':
            res = num2 !== 0 ? num1 % num2 : "Cannot divide by zero";
            break;
        case 'power':
            res = Math.pow(num1, num2);
            break;
        case 'square':
            res = num1 * num1;
            break;
        case 'sqrt':
            res = num1 >= 0 ? Math.sqrt(num1) : "Invalid for negative";
            break;
    }

    document.getElementById('result').innerText = "Result: " + res;
}
</script>

</body>
</html>
```

OUTPUT



E/ CEC

JSON (JavaScript Object Notation) is a lightweight data format used to store and exchange data. JavaScript provides built-in methods to convert between JSON and JavaScript objects:

- **JSON.parse()** → Converts JSON text to a JavaScript object.
- **JSON.stringify()** → Converts a JavaScript object to JSON text.

Dates in JSON are stored as strings, so we must manually convert them to JavaScript Date objects.

CSV (Comma-Separated Values) is a text format for tabular data. Converting JSON to CSV requires extracting object values into a comma-separated string, and CSV to JSON requires splitting the string and mapping values.

The **crypto.createHash()** method (Node.js) generates a hash value (like SHA256, MD5) for a string.

7. Develop JavaScript program (with HTML/CSS) for:

a) Converting JSON text to JavaScript Object

b) Convert JSON results into a date

c) Converting From JSON To CSV and CSV to JSON

d) Create hash from string using crypto.createHash() method

<u>Json-tools.html</u>

```
<!DOCTYPE html>
<html>
<head>
   <title>JSON Utilities</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         margin: 20px;
         background-color: #f5f5f5;
      }
      textarea, input, button {
         width: 100%;
         padding: 8px;
         margin: 6px 0;
      }
      textarea {
         height: 100px;
      }
      .section {
         background: white;
         padding: 15px;
         margin-bottom: 20px;
         border-radius: 6px;
         box-shadow: 0 0 5px rgba(0,0,0,0.1);
      }
      pre {
         background: #eee;
         padding: 10px;
         white-space: pre-wrap;
```

CSE/ CEC

```
        }
        button {
            background: #007bff;
            border: none;
            color: white;
            cursor: pointer;
        }
        button:hover {
            background: #0056b3;
        }
    </style>
</head>
<body>

<h1>JSON Utilities</h1>

<!-- a) JSON to JavaScript Object -->
<div class="section">
    <h2>a) JSON Text → JavaScript Object</h2>
    <textarea id="jsonInput"
placeholder='{"name":"Alice","age":25,"skills":["JavaScript","HTML","CSS"]}'></textarea>
    <button onclick="convertToObject()">Convert</button>
    <pre id="jsObjectOutput"></pre>
</div>

<!-- b) Convert JSON Date -->
<div class="section">
    <h2>b) Convert JSON Date String → Date Object</h2>
    <textarea id="jsonDateInput" placeholder='{"event":"Meeting","date":"2025-08-12T10:30:00Z"}'></textarea>
    <button onclick="convertDate()">Convert Date</button>
    <pre id="dateOutput"></pre>
</div>

<!-- c) JSON ↔ CSV -->
<div class="section">
    <h2>c) JSON ↔ CSV</h2>
    <textarea id="jsonCsvInput" placeholder='[{"name":"Alice","age":25},{"name":"Bob","age":30}]'></textarea>
    <button onclick="jsonToCsv()">JSON to CSV</button>
    <button onclick="csvToJson()">CSV to JSON</button>
    <pre id="csvJsonOutput"></pre>
</div>

<!-- d) Hash from String -->
<div class="section">
    <h2>d) Create Hash from String (Browser Web Crypto API)</h2>
    <input type="text" id="hashInput" placeholder="Enter text to hash">
    <button onclick="createHash()">Generate SHA-256 Hash</button>
    <pre id="hashOutput"></pre>
</div>

<script>
    // a) JSON to JavaScript Object
    function convertToObject() {
        try {
            let jsonText = document.getElementById("jsonInput").value;
            let obj = JSON.parse(jsonText);
```

CSE/CEC

```
        document.getElementById("jsObjectOutput").textContent = JSON.stringify(obj, null, 2);
    } catch (e) {
        document.getElementById("jsObjectOutput").textContent = "Invalid JSON!";
    }
}

// b) Convert JSON Date
function convertDate() {
    try {
        let jsonText = document.getElementById("jsonDateInput").value;
        let obj = JSON.parse(jsonText);
        let date = new Date(obj.date);
        document.getElementById("dateOutput").textContent = date.toString();
    } catch (e) {
        document.getElementById("dateOutput").textContent = "Invalid JSON or Date!";
    }
}

// c) JSON to CSV
function jsonToCsv() {
    try {
        let jsonArray = JSON.parse(document.getElementById("jsonCsvInput").value);
        let keys = Object.keys(jsonArray[0]);
        let csv = keys.join(",") + "\n" +
            jsonArray.map(row => keys.map(k => row[k]).join(",")).join("\n");
        document.getElementById("csvJsonOutput").textContent = csv;
    } catch (e) {
        document.getElementById("csvJsonOutput").textContent = "Invalid JSON!";
    }
}

// c) CSV to JSON
function csvToJson() {
    try {
        let csv = document.getElementById("jsonCsvInput").value;
        let lines = csv.trim().split("\n");
        let keys = lines[0].split(",");
        let jsonArray = lines.slice(1).map(line => {
            let values = line.split(",");
            let obj = {};
            keys.forEach((key, i) => obj[key] = values[i]);
            return obj;
        });
        document.getElementById("csvJsonOutput").textContent = JSON.stringify(jsonArray, null, 2);
    } catch (e) {
        document.getElementById("csvJsonOutput").textContent = "Invalid CSV!";
    }
}

// d) Hash from String (Web Crypto API)
async function createHash() {
    let text = document.getElementById("hashInput").value;
    if (!text) {
        document.getElementById("hashOutput").textContent = "Please enter some text!";
        return;
    }
```

CSE/CEC

```
        let encoder = new TextEncoder();
        let data = encoder.encode(text);
        let hashBuffer = await crypto.subtle.digest("SHA-256", data);
        let hashArray = Array.from(new Uint8Array(hashBuffer));
        let hashHex = hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
        document.getElementById("hashOutput").textContent = hashHex;
    }
</script>

</body>
</html>
```

OUTPUT

JSON Utilities

a) JSON Text → JavaScript Object

```
{"name":"Alice","age":25,"skills":["JavaScript","HTML","CSS"]}
```

Convert

```
{
  "name": "Alice",
  "age": 25,
  "skills": [
    "JavaScript",
    "HTML",
    "CSS"
  ]
}
```

b) Convert JSON Date String → Date Object

```
{"event":"Meeting","date":"2025-08-12T10:30:00Z"}
```

Convert Date

```
Tue Aug 12 2025 16:00:00 GMT+0530 (India Standard Time)
```

c) JSON ↔ CSV

```
[
  {"name":"Alice","age":25,"city":"Delhi"},
  {"name":"Bob","age":30,"city":"Mumbai"},
  {"name":"Charlie","age":28,"city":"Bangalore"}
]
```

JSON to CSV

CSV to JSON

```
name,age,city
Alice,25,Delhi
Bob,30,Mumbai
Charlie,28,Bangalore
```

d) Create Hash from String (Browser Web Crypto API)

Hello World!

Generate SHA-256 Hash

```
7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069
```

c) JSON ↔ CSV

```
name,age,city
Alice,25,Delhi
Bob,30,Mumbai
Charlie,28,Bangalore
```

JSON to CSV

CSV to JSON

```
[
  {
    "name": "Alice",
    "age": "25",
    "city": "Delhi"
  },
  {
    "name": "Bob",
    "age": "30",
    "city": "Mumbai"
  },
  {
    "name": "Charlie",
    "age": "28",
    "city": "Bangalore"
  }
]
```

PHP

Full Form: PHP originally stood for *Personal Home Page*, but now it's a recursive acronym for **PHP: Hypertext Preprocessor**.

Type: Server-side scripting language.

Purpose: To create dynamic, interactive web pages.

Main Features of PHP

- **Open Source** → Free to use and supported by a large community.
- **Platform Independent** → Works on Windows, Linux, macOS, etc.
- **Embedded in HTML** → You can write PHP code inside HTML pages.
- **Database Support** → Works with MySQL, PostgreSQL, Oracle, etc.
- **Cross-Browser Output** → The output is plain HTML, so it works on all browsers.
- **Server-Side Execution** → Code is executed on the server, and the browser receives only the HTML output.

CSE/

PHP Syntax Basics

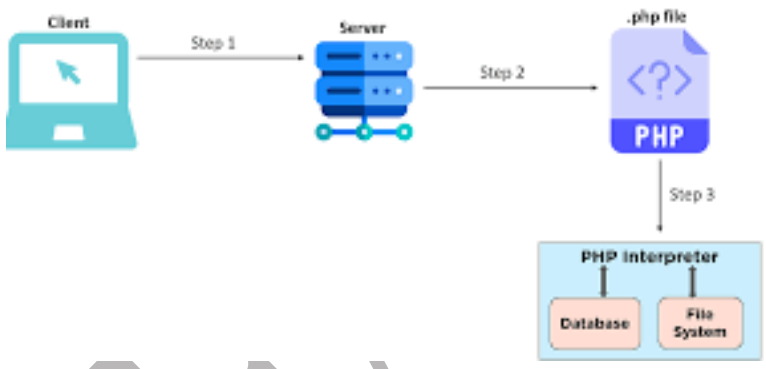
- PHP code is written inside:

```
<?php
// Your PHP code here
?>
```

- Statements end with a semicolon ;.
- Variables start with $ (e.g., $name = "John";).
- Comments can be:

```
// Single line comment
# Single line comment
/* Multi-line comment */
```

How PHP Works



1. A user requests a .php page from a browser.
2. The request is sent to the **web server** (e.g., Apache).
3. The server passes the file to the **PHP interpreter**.
4. PHP executes the script (may interact with a database, files, etc.).
5. The output (usually HTML) is sent back to the browser.
6. The browser **never sees the PHP code**, only the result.

Common Uses of PHP

- Building **dynamic websites**.
- Creating **form handling** systems.
- **Reading/writing files** on the server.
- **Database operations** (CRUD: Create, Read, Update, Delete).
- Session and cookie management.
- Generating **JSON, XML** for APIs.

Example of PHP embedded in HTML

```
<!DOCTYPE html>
<html>
<body>
  <h1>Welcome</h1>
  <?php
    $name = "Shruthi";
    echo "<p>Hello, $name! This page is powered by PHP.</p>";
  ?>
</body>
</html>
```

Steps to install and run xampp:

1. Download and install XAMPP from: https://www.apachefriends.org
2. Open the folder where XAMPP is installed (typically C:\xampp). Inside that, open the htdocs folder:
3. Place your PHP files (e.g., visitor.php, counter.txt, or sort_students.php) in this folder.
4. Open **XAMPP Control Panel** (search "XAMPP" in Start Menu).
5. Click **Start** next to **Apache**. Make sure it turns green and says "Running".

8. a. Develop a PHP program (with HTML/CSS) to keep track of the number of visitors visiting the web page and to display this count of visitors, with relevant headings.

Steps to Run

1. Open XAMPP Control Panel → Start Apache.
2. Save the above code in C:\xampp\htdocs\visitor.php.
3. Create a blank file named counter.txt in the same folder, and set its content to 0.
4. Open browser → visit**: http://localhost/visito.php**
5. Each refresh will increase the visitor count.

visitor.php

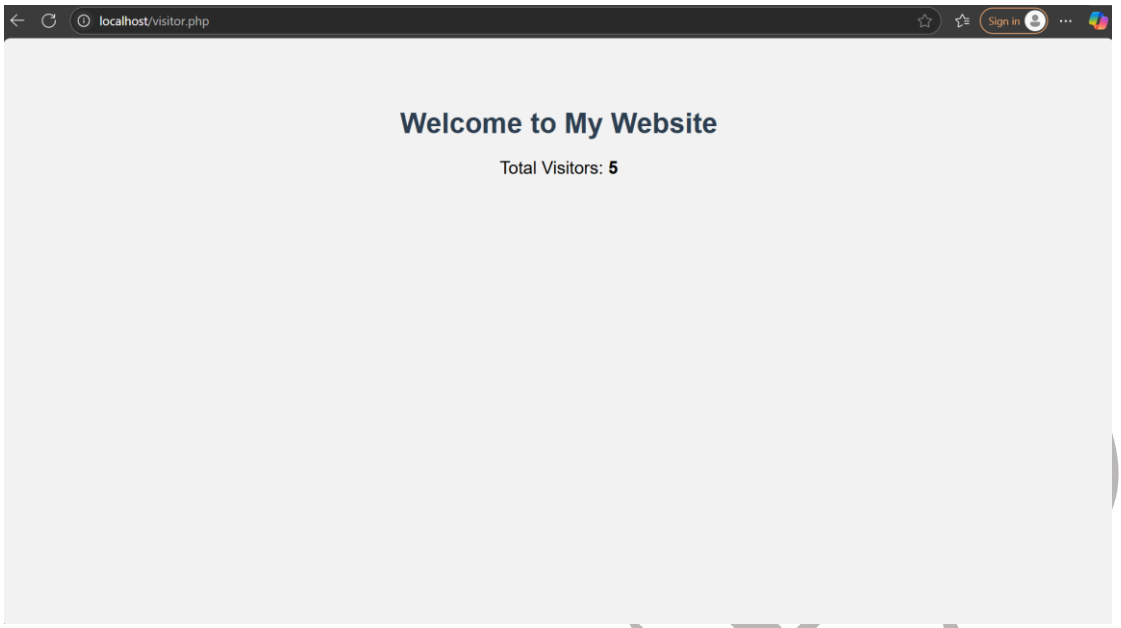
```
<?php
// File to store the visitor count
$counter_file = "counter.txt";

// If file doesn't exist, create it with value 0
if (!file_exists($counter_file)) {
   file_put_contents($counter_file, 0);
}

// Read the current count
```

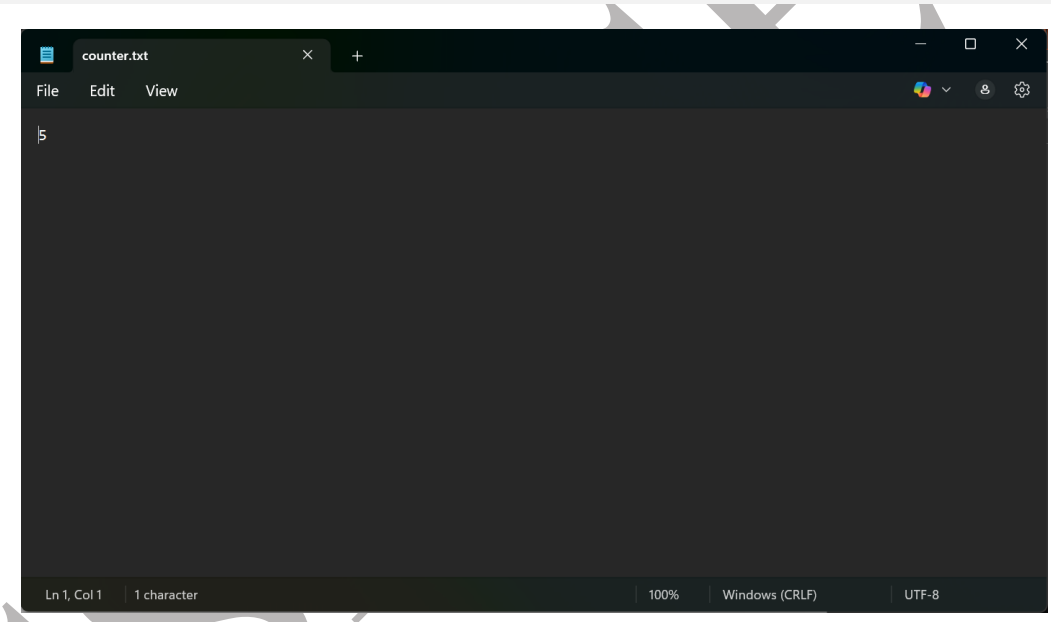
CSE/CEC

```
$counter = (int)file_get_contents($counter_file);

// Increase count by 1
$counter++;

// Save the updated count back to file
file_put_contents($counter_file, $counter);
?>
<!DOCTYPE html>
<html>
<head>
   <title>Visitor Counter</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         background-color: #f4f8fb;
         text-align: center;
         padding: 50px;
      }
      h1 {
         color: #333;
      }
      .counter-box {
         display: inline-block;
         padding: 20px;
         background-color: #ffffff;
         border-radius: 10px;
         box-shadow: 0px 0px 10px rgba(0,0,0,0.1);
      }
      .count {
         font-size: 40px;
         font-weight: bold;
         color: #007BFF;
      }
   </style>
</head>
<body>
   <h1>Welcome to My Website</h1>
   <div class="counter-box">
      <p>You are visitor number:</p>
      <div class="count"><?php echo $counter; ?></div>
   </div>
</body>
</html>
```

CSE/ CEC

OUTPUT


localhost/visitor.php
Sign in
Welcome to My Website
Total Visitors: 5

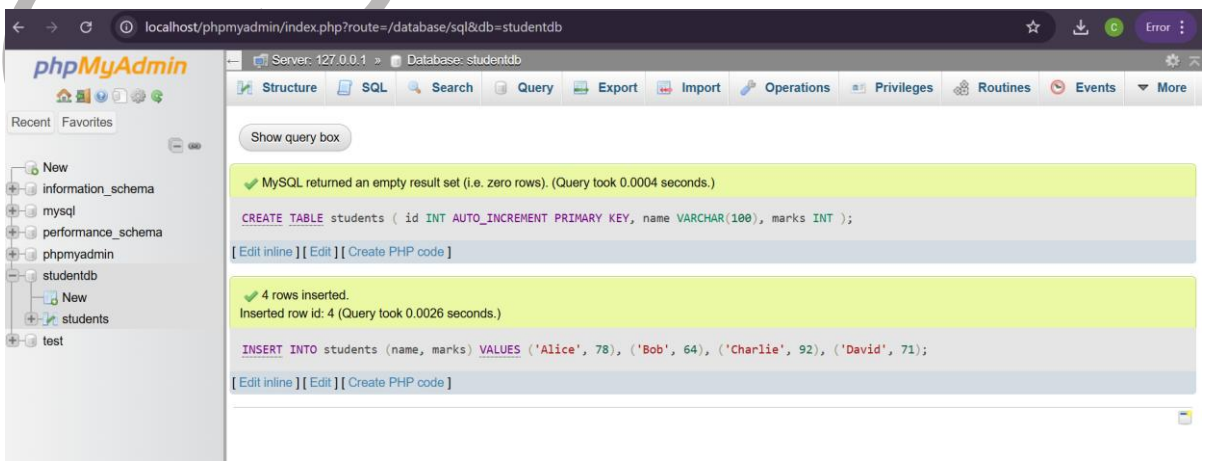

counter.txt
File
Edit
View
5
Ln 1, Col 1
1 character
100%
Windows (CRLF)
UTF-8

8b. Develop a PHP program (with HTML/CSS) to sort the student records which are stored in the database using selection sort.

1. **Start Apache and MySQL in XAMPP**
   - Open the **XAMPP Control Panel**
   - Click **Start** next to: **Apache, MySQL**
   - Make sure both are green (running).
2. **Create Database and Table**
   - **Open phpMyAdmin**
     1. In your browser, go to: http://localhost/phpmyadmin
     2. On the **left panel**, click **New**
     3. Under "Create database", enter: studentdb
     4. Choose **Collation** (or leave default utf8mb4_general_ci)
     5. Click **Create**
     6. If successful, you'll see: "Database studentdb has been created."
3. **Create Table and Insert Data**
   - Click on studentdb in the left sidebar
   - Click the **SQL** tab
   - Write the following SQL:

```
CREATE TABLE students (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100),
        marks INT);
INSERT INTO students (name, marks) VALUES
('Alice', 78),
('Bob', 64),
('Charlie', 92),
('David', 71);
```

4. Click **Go**- You should now see a table called students with 4 rows.



CSE/CEC

5. **Save PHP File to htdocs.**
   - Save the PHP file as: C:\xampp\htdocs\sort.php

sort.php:

```
<?php
// Database connection settings
$servername = "localhost";
$username = "root"; // default XAMPP username
$password = "";     // default XAMPP password (empty)
$dbname = "studentdb"; // your database name

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Fetch student data from database
$sql = "SELECT * FROM students";
$result = $conn->query($sql);

$students = [];
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        $students[] = $row;
    }
}

// Selection sort by marks (ascending order)
$n = count($students);
for ($i = 0; $i < $n - 1; $i++) {
    $minIndex = $i;
    for ($j = $i + 1; $j < $n; $j++) {
        if ($students[$j]['marks'] < $students[$minIndex]['marks']) {
            $minIndex = $j;
        }
    }
    // Swap
    $temp = $students[$i];
    $students[$i] = $students[$minIndex];
    $students[$minIndex] = $temp;
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Student Records Sorted by Marks</title>
    <style>
        body {
```

CSE/ CEC

```
            font-family: Arial, sans-serif;
            background-color: #f8f9fa;
            text-align: center;
        }
        table {
            margin: 20px auto;
            border-collapse: collapse;
            width: 50%;
        }
        th, td {
            border: 1px solid #ddd;
            padding: 8px;
        }
        th {
            background-color: #4CAF50;
            color: white;
        }
        tr:nth-child(even) {background-color: #f2f2f2;}
    </style>
</head>
<body>
    <h2>Student Records Sorted by Marks (Selection Sort)</h2>
    <table>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Marks</th>
        </tr>
        <?php foreach ($students as $student): ?>
        <tr>
            <td><?php echo $student['id']; ?></td>
            <td><?php echo $student['name']; ?></td>
            <td><?php echo $student['marks']; ?></td>
        </tr>
        <?php endforeach; ?>
    </table>
</body>
</html>
```

6. **Open browser and type: http://localhost/sort.php**

CSE/ CEC

OUTPUT

localhost/sort.php

Student Records Sorted by Marks (Selection Sort)

ID	Name	Marks
2	Bob	64
4	David	71
1	Alice	78
3	Charlie	92

Introduction to jQuery

jQuery is a fast, lightweight, and feature-rich **JavaScript library** designed to simplify HTML document traversal, manipulation, event handling, animation, and AJAX interactions.

It was created by **John Resig** in 2006 with the motto **"Write less, do more"**, meaning that jQuery lets you perform complex JavaScript tasks using short and simple code.

Key Features of jQuery

1. **Simplified Syntax** – Shorter and easier code compared to vanilla JavaScript.
2. **Cross-browser Compatibility** – Works consistently across major browsers.
3. **DOM Manipulation** – Easily add, remove, or change HTML elements and attributes.
4. **Event Handling** – Attach events like click, hover, etc., with minimal code.
5. **Animations & Effects** – Built-in methods for visual effects.
6. **AJAX Support** – Load data from the server without reloading the page.
7. **Extensibility** – Supports plugins to extend functionality.

Advantages of jQuery

- Reduces code length significantly.
- Handles browser compatibility issues.
- Provides a large number of ready-to-use functions.
- Has a vast plugin ecosystem.
- Well-documented and widely supported.

CSE/CEC

Basic Syntax

$(selector).action();

- **$** → jQuery object.
- **selector** → Finds HTML elements (e.g., #id, .class, tag).
- **action()** → Method to perform an operation (e.g., .hide(), .append()).

Example:

$("#myPara").hide(); // Hides the element with id="myPara"

Including jQuery in a Web Page

You can include jQuery by:

1. **CDN** (Content Delivery Network):

   <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

2. **Local File** (Downloaded version):

   <script src="jquery.min.js"></script>

9. Develop jQuery script (with HTML/CSS) for:
a. Appends the content at the end of the existing paragraph and list.
b. Change the state of the element with CSS style using animate() method
c. Change the color of any div that is animated.

a. jQuery .append()

- **Purpose:** Inserts specified content at the **end** of the selected elements.
- **Usage:**
- $("#para").append(" New text here.");
- $("#list").append("<li>New item</li>");
- **Note:** The new content becomes part of the existing element.

b. jQuery .animate()

- **Purpose:** Creates **custom animations** for CSS properties.
- **Syntax:**
- $(selector).animate({property: value}, duration);
- **Example:**
- $("#box").animate({left: '250px', height: '200px'}, 1000);
- **Note:** Only numeric CSS properties can be animated (e.g., width, height, opacity, position values).

c. Changing Color of an Animated Element

- **Issue:** .animate() **cannot directly animate colors** because color values are not numeric.
- **Solution:** Use .css() to instantly change color before/after animation OR use the **jQuery UI library** which supports color animation.
- **Example:**
- $("#box").css("background-color", "blue");

CSE/CEC

Jquery_lab9.html

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Append and Animate Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
    }
    #box {
      width: 100px;
      height: 100px;
      background-color: red;
      position: relative;
      margin: 20px auto;
    }
    p, ul {
      width: 300px;
      margin: auto;
      text-align: left;
    }
    button {
      padding: 10px;
      margin: 5px;
      background: #4CAF50;
      color: white;
      border: none;
      cursor: pointer;
    }
    button:hover {
      background: #45a049;
    }
  </style>
  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>

  <h2>jQuery Append, Animate & Change Color</h2>

  <p id="para">This is an existing paragraph.</p>
  <ul id="list">
    <li>Item 1</li>
    <li>Item 2</li>
  </ul>

  <div id="box"></div>

  <button id="appendBtn">Append Content</button>
  <button id="animateBtn">Animate & Change Color</button>
```

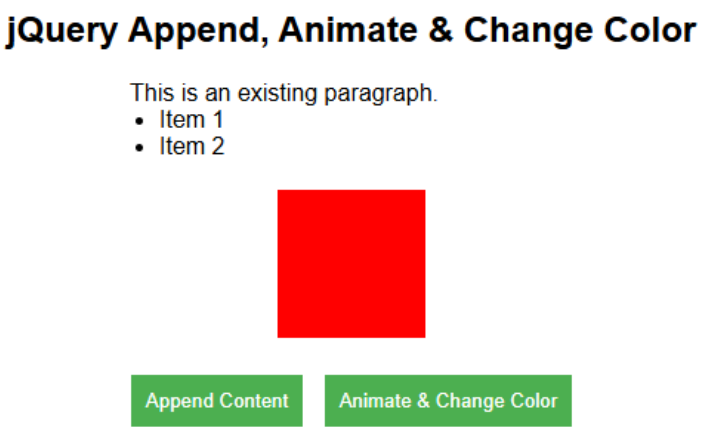
CSE/ CEC

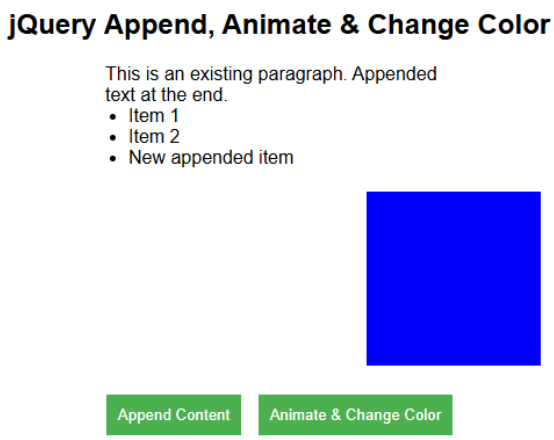
```
<script>
  $(document).ready(function(){
    // Append content
    $("#appendBtn").click(function(){
      $("#para").append(" Appended text at the end.");
      $("#list").append("<li>New appended item</li>");
    });

    // Animate box and change color
    $("#animateBtn").click(function(){
      $("#box").animate({
        left: '150px',
        height: '150px',
        width: '150px'
      }, 1000, function(){
        // After animation completes
        $(this).css("background-color", "blue");
      });
    });
  });
</script>

</body>
</html>
```

How It Works

1. **Append Button** → Adds extra text to paragraph and an extra list item.
2. **Animate Button** → Moves and resizes the red box, then changes its color to blue after animation ends.

OUTPUT





CE/CEC

AJAX

What is AJAX?

- AJAX stands for Asynchronous JavaScript And XML.
- It's not a programming language — it's a technique to send and receive data in the background without reloading the whole web page.
- Despite its name, AJAX works with JSON, text, HTML, and XML — not only XML.

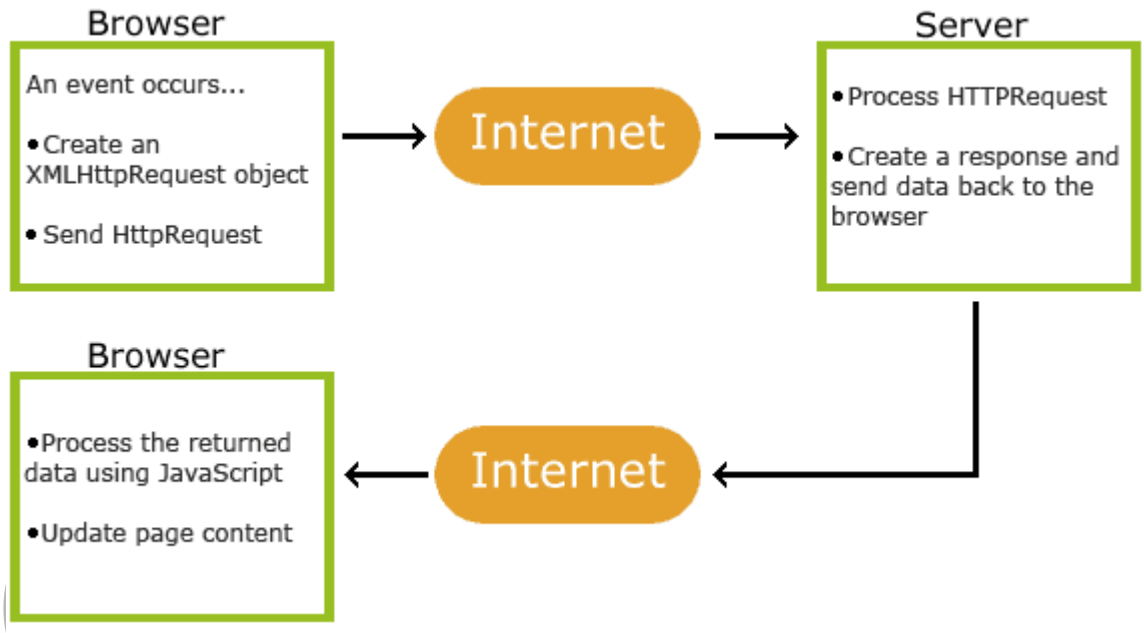
Why use AJAX?

- Faster user experience (no full page reload).
- Real-time updates (e.g., search suggestions, chat apps).
- Reduce server load (only request what's needed).

How AJAX Works

Think of it as a messenger between the browser and the server:

1. A user triggers an event (click, scroll, page load).
2. JavaScript creates an AJAX request.
3. The request is sent to the server.
4. The server processes it and sends back data (text, JSON, XML…).
5. JavaScript updates part of the web page without refreshing.



Common AJAX Methods

You can do AJAX in two main ways:

a) Without jQuery — Using XMLHttpRequest or fetch()

```
let xhr = new XMLHttpRequest();
xhr.open("GET", "data.txt", true);
xhr.onload = function() {
   if (xhr.status === 200) {
      document.getElementById("output").innerText = xhr.responseText;
   }
};
xhr.send();
```

b) With jQuery

```
$.ajax({
   url: "data.txt",
   method: "GET",
   success: function(data) {
      $("#output").text(data);
   }
});
```

5. Data Formats in AJAX

- **Text** → Simple text files
- **JSON** → { "name": "Alice", "age": 22 } (most common today)
- **XML** → <person><name>Alice</name></person> (less common now)
- **HTML** → Fragments of HTML content

6. Key AJAX Methods in jQuery

Method	Description
$.ajax()	Most flexible AJAX method, lets you fully configure the request.)
$.get()	Sends a simple HTTP GET request to fetch data.
$.post()	Sends a simple HTTP POST request with data.
$.getJSON()	Sends a GET request and automatically parses the JSON response.
$.load()	Loads HTML content from a server directly into an element.

.

CSE/CEC

10. Develop a JavaScript program with Ajax (with HTML/CSS) for:
a. Use ajax() method (without Jquery) to add the text content from the text file by sending ajax request.
b. Use ajax() method (with Jquery) to add the text content from the text file by sending ajax request.

Step 1 : Folder Setup in XAMPP

a) Go to your **XAMPP htdocs** folder:
   i. C:\xampp\htdocs\
b) Create a new folder for this project: ajax-demo
c) Inside ajax-demo, create these files:
   i. index.html
   ii. text1.txt
   iii. text2.txt
   iv. data.json

text1.txt
This is the content loaded from text1.txt using Plain JavaScript AJAX.
text2.txt
This is the content loaded from text2.txt using jQuery AJAX.
data.json

```
{
  "name": "Alice",
  "age": 22,
  "city": "Bangalore"
}
```

index.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>AJAX Example - Text & JSON</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px;
    }
    button {
      padding: 10px 15px;
      margin: 5px;
      cursor: pointer;
    }
    #output {
      margin-top: 20px;
      padding: 15px;
      border: 1px solid #ccc;
      min-height: 50px;
    }
```

CSE/ CEC

```
        table {
            border-collapse: collapse;
            margin-top: 10px;
        }
        th, td {
            padding: 8px 12px;
            border: 1px solid #444;
        }
        th {
            background: #f0f0f0;
        }
    </style>
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>

<h2>AJAX Demo</h2>

<button id="btnLoadTextJS">a. Load Text (Plain JS)</button>
<button id="btnLoadTextJQ">b. Load Text (jQuery)</button>
<button id="btnLoadJSON">c. Load JSON (getJSON)</button>
<button id="btnParseJSON">d. Parse JSON (parseJSON)</button>

<div id="output">Output will appear here...</div>

<script>
// a. Plain JavaScript AJAX
document.getElementById('btnLoadTextJS').addEventListener('click', function() {
    var xhr = new XMLHttpRequest();
    xhr.open('GET', 'text1.txt', true);
    xhr.onload = function() {
        if (xhr.status === 200) {
            document.getElementById('output').innerText = xhr.responseText;
        } else {
            document.getElementById('output').innerText = 'Error loading text.';
        }
    };
    xhr.send();
});

// b. jQuery AJAX
$('#btnLoadTextJQ').click(function() {
    $.ajax({
        url: 'text2.txt',
        method: 'GET',
        success: function(data) {
            $('#output').text(data);
        },
```

CSE/ CEC

```
        error: function() {
            $('#output').text('Error loading text.');
        }
    });
});

// c. Load JSON with getJSON()
$('#btnLoadJSON').click(function() {
    $.getJSON('data.json', function(data) {
        $('#output').html(
            `<strong>Name:</strong> ${data.name}<br>
             <strong>Age:</strong> ${data.age}<br>
             <strong>City:</strong> ${data.city}`
        );
    }).fail(function() {
        $('#output').text('Error loading JSON.');
    });
});

// d. Parse JSON with parseJSON()
$('#btnParseJSON').click(function() {
    $.ajax({
        url: 'data.json',
        method: 'GET',
        dataType: 'text', // force text to parse manually
        success: function(response) {
            var data = $.parseJSON(response);
            var html = `<table>
                        <tr><th>Name</th><th>Age</th><th>City</th></tr>
                        <tr><td>${data.name}</td><td>${data.age}</td><td>${data.city}</td></tr>
                      </table>`;
            $('#output').html(html);
        },
        error: function() {
            $('#output').text('Error parsing JSON.');
        }
    });
});
</script>

</body>
</html>
```

Step 2 – Start Apache Server

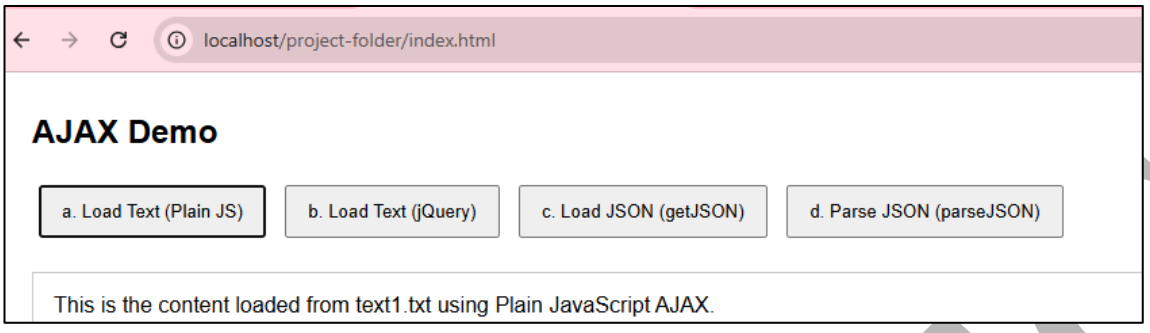
1. Open **XAMPP Control Panel**.
2. Start the **Apache** module.

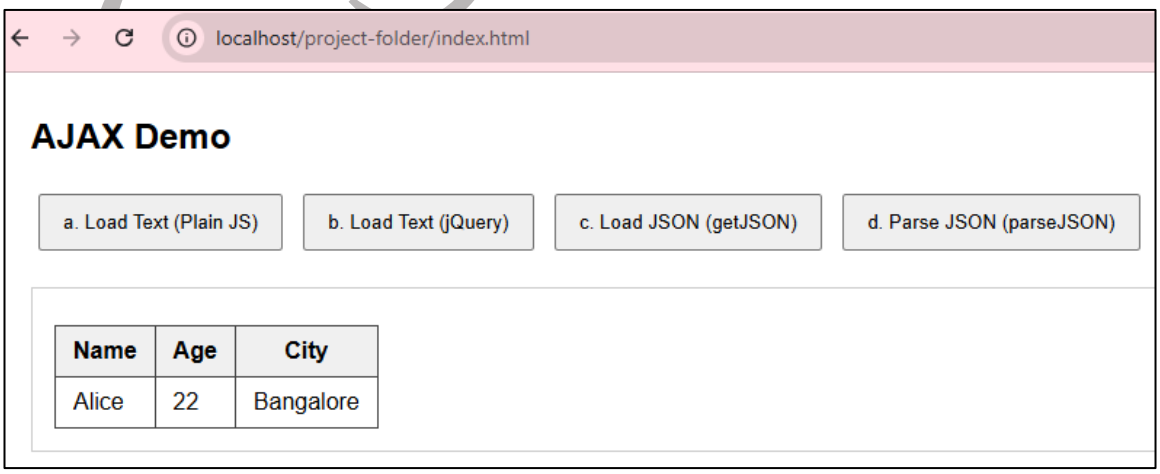
CSE/ CEC

Step 4 – Run the Program

1. Open your browser.
2. Type in the following URL: http://localhost/ajaxdemo/index.html
3. Click each button to see different outputs.

OUTPUT





VIVA QUESTIONS WITH ANSWERS

1. What is web development?
Web development is the process of building, creating, and maintaining websites, involving both frontend and backend technologies.

2. Name the two major areas of web development.
Frontend development and backend development.

3. Which languages are mainly used in frontend development?
HTML, CSS, and JavaScript.

4. Give two examples of backend programming languages from the manual.
PHP and Node.js (JavaScript).

5. What is the function of a database in web development?
To store, organize, and manage website data like user details, products, and transactions.

6. Name any two data exchange formats.
JSON and XML.

7. What does HTTP stand for?
HyperText Transfer Protocol.

8. What is the difference between HTTP and HTTPS?
HTTPS is a secure version of HTTP, using SSL/TLS encryption.

9. What is the role of DNS?
It converts human-friendly domain names into IP addresses.

10. What is HTML used for?
To create and structure the content of web pages.

11. What are HTML attributes?
Extra information about an element, specified inside the opening tag.

12. Give an example of a void element.

 or <hr>.

13. What are structural tags in HTML?
Tags like <html>, <head>, and <body> that define the basic structure of the page.

14. Which tag is used for the largest heading?
<h1>.

15. How is a hyperlink created in HTML?
Using <a href="URL">Link text</a>.

16. Which tag is used for an unordered list?
<ul>.

17. What does rowspan do in a table?
Merges multiple rows into one cell.

18. What does colspan do in a table?
Merges multiple columns into one cell.

CSE/CEC

19. In a table, how are lab hours highlighted?
Using a specific CSS class with a different background color.

20. What does CSS stand for?
Cascading Style Sheets.

21. Name the three types of CSS.
Inline, internal, and external.

22. Which CSS selector targets a specific element ID?
ID selector (#idname).

23. What does tr:nth-child(even) do in CSS?
Applies styles to every even-numbered table row.

24. What is the purpose of a <form> tag?
To collect and submit user input.

25. Which attribute specifies the URL where form data is sent?
action.

26. Which attribute specifies the HTTP method for form submission?
method.

27. Give two form input types from the registration form example.
text and radio.

28. What is the use of <fieldset> and <legend>?
They group related form elements and provide a caption.

29. What are semantic HTML elements?
Tags that describe their meaning, such as <header> and <footer>.

30. Which semantic tag is used for navigation menus?
<nav>.

31. Which semantic tag is used for independent content blocks?
<article>.

32. In the newspaper example, what is <figcaption> used for?
To provide a caption for the <figure> element.

33. What is JavaScript mainly used for?
To add interactivity and dynamic behavior to web pages.

34. How is JavaScript included in HTML?
Using the <script> tag.

35. Which method is used to select an element by ID in JavaScript?
document.getElementById().

36. Name two JavaScript functions from the calculator example.
Math.sqrt() and Math.pow().

37. How is division by zero handled in the calculator example?
By checking if the divisor is not zero before dividing.

CSE/CEC

38. What does JSON stand for?
JavaScript Object Notation.

39. Which method converts JSON text to a JavaScript object?
JSON.parse().

40. Which method converts a JavaScript object to JSON text?
JSON.stringify().

41. In the JSON tools example, what does try...catch handle?
Invalid JSON input errors.

42. What is the purpose of CSV?
To store tabular data in plain text.

43. What is the crypto.subtle.digest() method used for ?
To generate a SHA-256 hash from a string.

44. What is PHP?
A server-side scripting language for creating dynamic web pages.

45. Where are PHP files placed when using XAMPP?
In the htdocs folder.

47. Which PHP function reads file contents?
file_get_contents().

48. Which PHP function writes data to a file?
file_put_contents().

50. How do you connect PHP to a MySQL database?
Using new mysqli(host, username, password, dbname).

51. What does $result->fetch_assoc() do?
Fetches a result row as an associative array.

52. What is jQuery?
A lightweight JavaScript library for simplified scripting.

53. How do you include jQuery from a CDN?
Using <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>.

54. Which jQuery method adds content to the end of an element?
.append().

55. Which jQuery method is used for animation?
.animate().

56. Can .animate() directly change colors in jQuery?
No, color animation needs jQuery UI or .css() changes.

57. In the jQuery example, when is the box color changed to blue?
After the animation completes, inside the callback function.

58. What is AJAX used for?
To update parts of a webpage without reloading it.

CSE/ CEC

59. Which JavaScript object is used for AJAX requests without jQuery?
XMLHttpRequest.

60. In the AJAX example, which method sends a GET request in plain JS?
xhr.open('GET', 'filename', true).

61. Which jQuery method is used to load JSON data?
$.getJSON().

62. Which jQuery method is used to manually parse JSON text?
$.parseJSON().

63. In AJAX, what does HTTP status code 200 mean?
The request was successful.

64. In AJAX, what does HTTP status code 404 mean?
File not found.

65. What is the readyState value when a request is complete?
4.

66. In the manual's AJAX example, where is the output displayed?
Inside the element with ID output.

67. Why is dataType: 'text' used before parsing JSON manually?
To prevent automatic parsing by jQuery.

68. What is the benefit of using external CSS in the examples?
It separates style from content and can be reused on multiple pages.

69. In the timetable example, how are even and odd rows colored differently?
Using tr:nth-child(even) and tr:nth-child(odd) selectors.

70. What is the function of <pre> in HTML?
To preserve spaces and line breaks exactly as in the code.

71. Which HTML element is used to highlight text in the example?
<mark>.

72. Which HTML element is used for quoting in Myfirstwebpage.html?
<blockquote>.

73. Which HTML element is used for inserting an image in the newspaper example?
<img>.

74. In the newspaper example, what is the purpose of <aside>?
To display related content such as advertisements.

75. What does © represent?
The copyright symbol.

?

CSE/CEC